Curse of Dimensionality

Contents

1. Where we are in feature engineering 2
2. What the curse of dimensionality is 2
3. An example: images of digits 3
    3.1 Measuring the effect 4
4. Why more dimensions cause trouble 5
    4.1 Finding a lost wallet 5
    4.2 The same data spreads thin 5
    4.3 Why this hurts models 7
5. The two problems 8
6. The solution: dimensionality reduction 9
7. Summary 11
8. Sources 11
9. Key terms 12

Where this fits: Pipeline map step 6 (Reduce dimensions). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	**6 Reduce**
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Unsupervised learning; Feature extraction; Feature selection.
- **Leads to:** PCA; K-nearest neighbours.
- **Compare with:** Superposition and nearly perpendicular directions.

1. Where we are in feature engineering

Key point: Two parts of feature engineering are left: feature selection and feature extraction. Both exist to fight the **curse of dimensionality** (G-520).

Of the four parts of feature engineering (see the four parts of feature engineering), feature transformation (missing values, encoding, scaling, outliers) and feature construction and splitting are done.

Two parts remain:

- **Feature selection** (G-768): keep only the useful **features** (G-772). Feature selection is usually done after a model is built, to check whether removing features helps the model.
- **Feature extraction** (G-762): build a few new features from the old ones. Its most important technique, PCA, comes next.

Both rest on one idea: the curse of dimensionality. This Note explains that idea, why it matters, and how we deal with it.

2. What the curse of dimensionality is

Key point: Every dataset has an optimal number of features. Past it, extra features do not help, and often make the model worse and slower.

In ML, each **feature** (G-772) (an input variable, one column of the data table) is called a dimension. Each **observation** (G-1374) is one record (one row), here one image of a handwritten digit (the running example of Section 3), and the **target** (G-1949) is the output we predict, here the digit (see dimensionality reduction). A dataset with 10 features is 10-dimensional, and one with 1,000 features is **high-dimensional** (G-897). Here "dimension" has the "dimensions of a vector" sense, not the number of axes of a tensor; a 1D tensor can be a 3-dimensional vector separates the two.

More features seem as if they should always help, since the model gets more information. In practice more features do not always help. Think of a detective handed a thousand pages of witness statements, most of them about the weather: the few useful clues get harder to find, not easier.

Every dataset has an **optimal number of features** (G-1397). Up to that number, each new feature improves the model. Beyond it:

- performance stops improving, and often gets worse;
- the model needs more computation and becomes more complex.

High-dimensional data is common. Two everyday sources are images, where every pixel is a feature, and text, where every word is a feature: an email spam classifier can easily work on 3,000 features.

These problems together are the **curse of dimensionality** (G-520): the problems that appear when data has too many dimensions (a name from Bellman; ESL §2.5).

3. An example: images of digits

> **Key point:** In images, every pixel is a **feature** (G-772). Many pixels, such as the ones at the edges, carry almost no information.

Image data shows the curse clearly: every pixel is one feature, and the blank edge pixels of MNIST (the classic set of handwritten-digit images) carry almost nothing (see **feature selection** (G-768)). Here we measure this on a smaller version, scikit-learn's digits dataset: 1,797 images of 8×8 pixels, so 64 features. Figure 1 shows one image and how much each pixel changes across all the images.

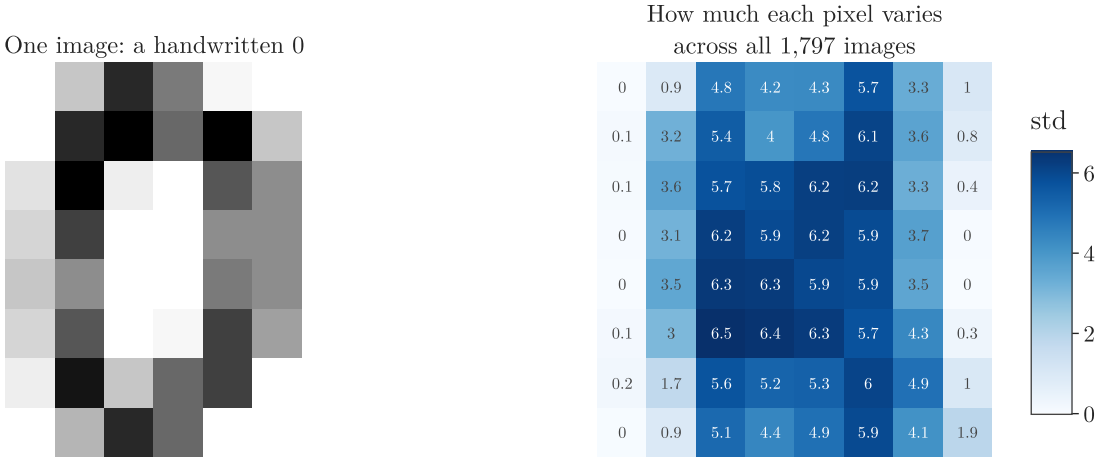


Figure 1: Edge pixels barely change from image to image

The digit is always drawn in the middle. The pixels at the left and right edges are almost always blank: 3 pixels never change at all, and 16 have a standard deviation (how far their values typically lie from their mean) below 1.

A pixel that is the same in every image cannot help tell one digit from another. Such a pixel is a feature the model has to process, with nothing to learn from.

3.1 Measuring the effect

> **Key point:** Accuracy rose to 96% with the useful pixels, then fell to 80% as we added useless **features** (G-772).

Figure 2 shows a real experiment with KNN, a classifier that labels a point by looking at its nearest neighbours. The score is **accuracy**, the share of test images whose digit the model names correctly.

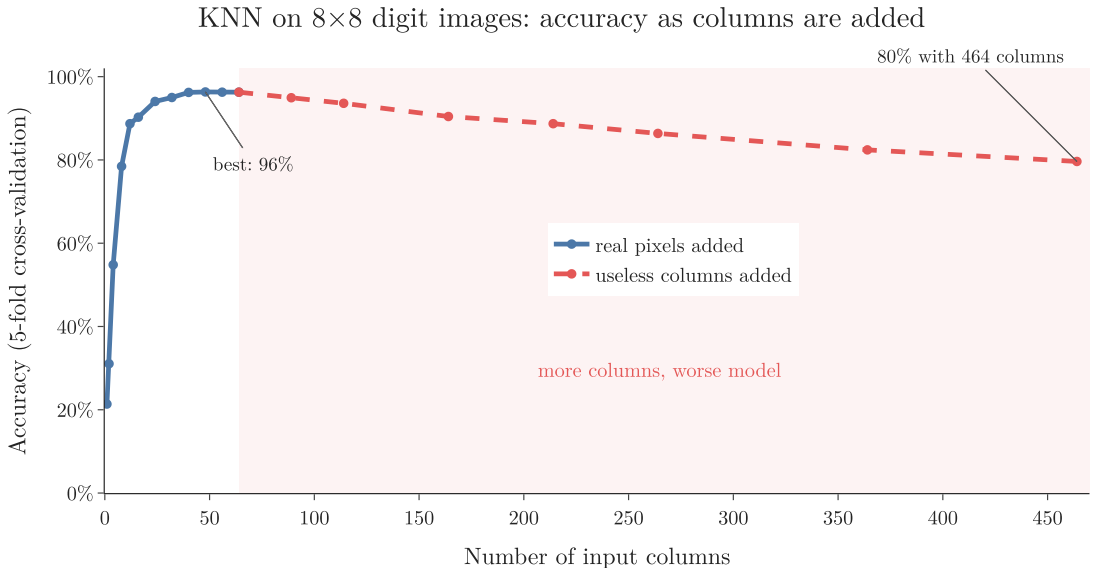


Figure 2: KNN accuracy as features are added (real measurement)

1. **Blue line:** we add the 64 pixels one group at a time, most useful first. Accuracy rises fast, from 21% with 1 pixel to 90% with 16 pixels. Accuracy levels off at 96% from about 40 pixels on.
2. **Red line:** we keep all 64 pixels and add features of random numbers. Accuracy drops steadily, to 80% with 400 extra features.

The blue line shows the **optimal number of features** (G-1397): after about 40 pixels, adding more brings nothing. The red line shows that useless features actively hurt.

> **Python:** Accuracy for different numbers of features (full code in the Notebook).

```
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

def score(Z):
    # average accuracy over 5 train/test splits
```

```
    return cross_val_score(KNeighborsClassifier(), Z, y, cv=5).mean()

score(X[:, order[:16]])                # best 16 pixels: 90%
score(np.hstack([X, noise[:, :400]])) # 64 pixels + 400 random: 80%
```

Extra: The blue line ranks the pixels by usefulness (mutual information, a score of how much a pixel tells about the digit) on all 1,797 images, including the images each cross-validation round later tests on. Ranking features with test data is a form of data leakage (G-535): the test images helped choose the features. Here it barely matters: ranking inside each training part instead gives 90% with 16 pixels and 96% with 40, the same as Figure 2 (only the 1-pixel score drops, from 21% to 17%; Notebook, leakage check). In a real project, choose features on the training data only.

Extra: **Cross-validation** (G-510), used in `cross_val_score`, splits the data into 5 parts and tests on each part in turn. Cross-validation gives a more reliable accuracy than a single train/test split. It is covered in detail later.

4. Why more dimensions cause trouble

Key point: As dimensions grow, the same data gets spread thinner and thinner, so points end up far from each other.

4.1 Finding a lost wallet

Key point: Each extra dimension makes the space to search much larger.

Suppose we lose our wallet. Where is it easiest to find?

1. **On a road** from the college gate to the building. The road is one-dimensional: we walk along it once and see everything.
2. **On the campus.** Now it can be anywhere on a two-dimensional area. The search takes much longer.
3. **In a multi-storey building.** Now it can be on any floor, in any room. Three dimensions make the search harder still.

The wallet is the same in each case. Only the number of dimensions changed, and with it the size of the space. Figure 3 in the next section draws the road, the campus and the building as grids of cells.

4.2 The same data spreads thin

Key point: With 5 bins per **feature** (G-772), each new feature multiplies the number of cells by 5, while the number of points stays the same.

Figure 3 applies the wallet idea to data. We take the same 20 points and split each feature into 5 bins (5 equal ranges of values).

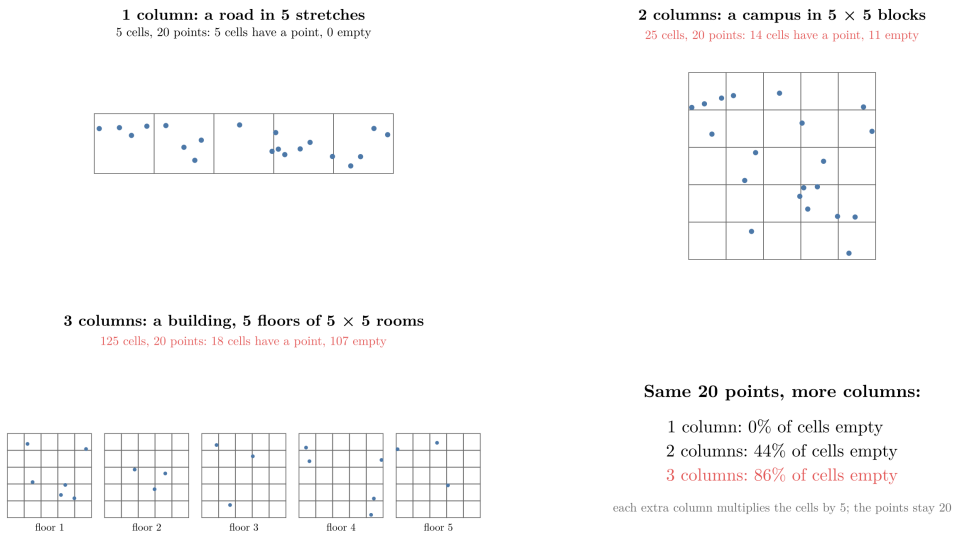


Figure 3: The same 20 points in 1, 2 and 3 features

Count the cells first. One feature in 5 bins gives 5 cells. A second feature splits each of them into 5 again, so 2 features give:

$$5 \times 5 = 25 \text{ cells}$$

and 3 features give:

$$5 \times 5 \times 5 = 125 \text{ cells}$$

Meanwhile there are still only 20 points.

Features	Like	Cells	Empty cells
1	a road in 5 stretches	5	0 (0%)
2	a campus of 5 × 5 blocks	25	11 (44%)
3	a building of 5 floors, 5 × 5 rooms each	125	107 (86%)

The number of cells grows as 5^d, where d is the number of features. Twenty points are enough to fill a road, but they leave a building almost empty.

Data where most of the space is empty is called **sparse**. In sparse data, every point is far from the others.

This "sparse" has a different meaning from the sparse data of **normalization** (G-1349), where sparse means a table whose values are mostly zeros. Here it means a space that is mostly empty.

4.3 Why this hurts models

> **Key point:** Many algorithms rely on distances between points. When every point is far from every other, distances stop being useful.

Many algorithms, such as KNN, make a prediction by finding the points closest to a new point. Finding the closest points only works if "close" means something.

In high dimensions, every point is far from every other point. The nearest neighbour is not really near, so the prediction is based on points that are not similar. These dissimilar neighbours are why the KNN accuracy in Figure 2 fell as we added useless **features** (G-772).

> **Extra:** We checked this on the red line of Figure 2. For every image we found its nearest other image and asked whether it shows the same digit (Notebook, after the accuracy table):
>
> | Features | Nearest neighbour shows the same digit | Nearest distance / average distance |
> |---|---|---|
> | 64 | 98.8% | 0.34 |
> | 164 | 91.0% | 0.76 |
> | 464 | 72.5% | 0.89 |
>
> With more useless features, the nearest image is almost as far away as an average one, and more and more often it shows a different digit. KNN votes with these neighbours, so its accuracy falls with them.

> **Extra:** In very high dimensions, distances also become almost all the same: the farthest point is hardly farther than the nearest (Beyer et al. 1999). Figure 4 measures the distance from one random point to 500 others. With 2 features, the farthest point is 67 times farther than the nearest; with 1,000 features, only 1.1 times. In Figure 4, each panel is a histogram: the horizontal axis is each distance divided by the average distance, and the bar height is the share of the 500 points at that distance. With 2 features the bars spread from 0 to 2; with 1,000 features they all pile up at 1. When everything is about equally far, "nearest" carries almost no information.

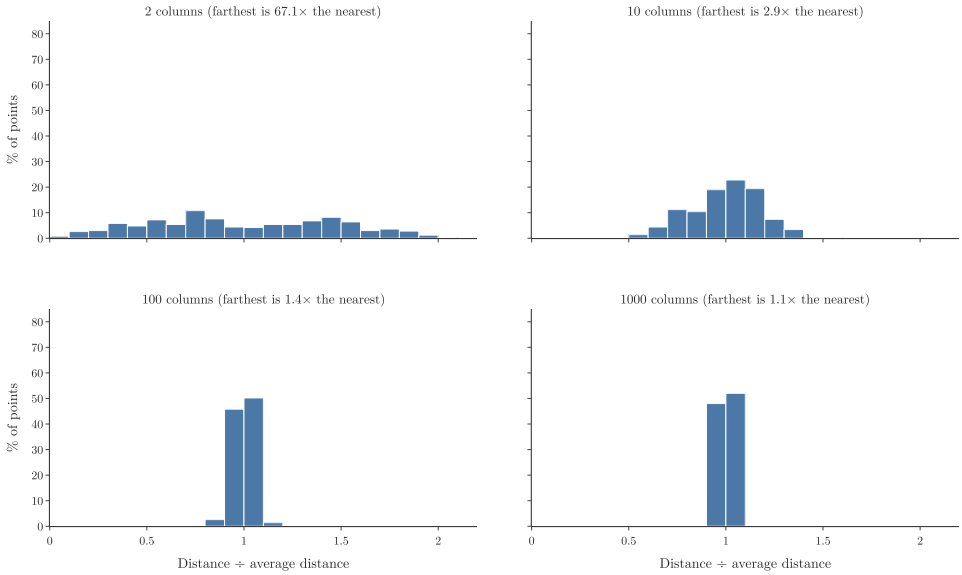


Figure 4: Distances from one point to 500 others, in 2 to 1,000 features

5. The two problems

> **Key point:** Too many dimensions cause two problems: lower performance and more computation.

1. **Performance decreases.** Useless **features** (G-772) spread the data thin and push the truly similar points apart (Section 4.3). In Figure 2, accuracy fell from 96% to 80%.
2. **Computation increases.** Every extra feature is more data to store and more numbers to process in every step. In the same experiment, scoring the model with 464 features took about 1.6 to 2.4 times as long as with 64, depending on the run.

Figure 5 draws both problems on the red-line experiment of Figure 2. As random features are added, the share of images whose nearest other image shows the same digit falls from 98.8 to 72.5 percent. The time to score the model rises overall, from about 0.028 seconds with 64 features to 0.045 seconds with 464; the small dip between 64 and 164 features is timing noise of one machine.

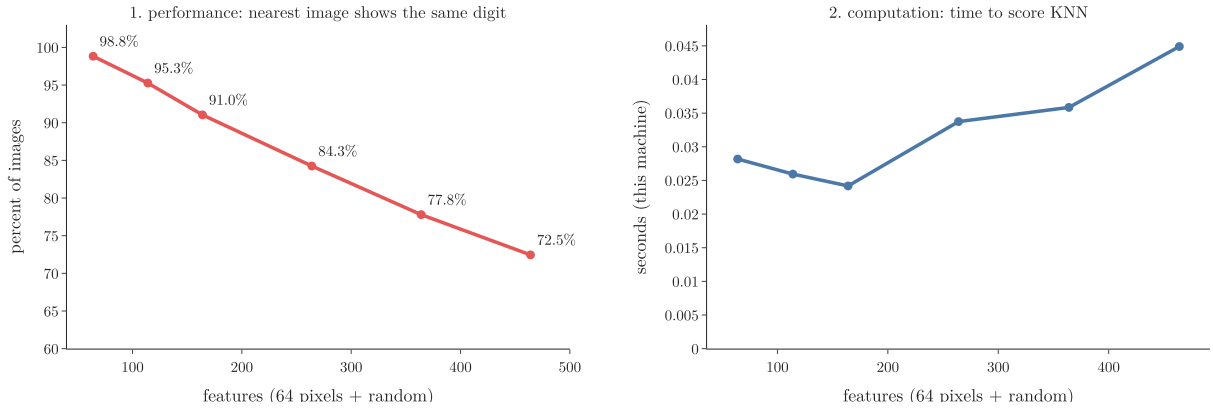


Figure 5: The digits data with useless random features added. Left: the share of images whose nearest other image shows the same digit. Right: the time to score KNN with 5-fold cross-validation (best of 3 runs; machine dependent).

The fix for both is to bring the number of features down to the optimal number.

6. The solution: dimensionality reduction

> **Key point:** Dimensionality reduction lowers the number of **features** (G-772). Dimensionality reduction comes in two kinds: feature selection keeps the best features; feature extraction builds new ones.

Dimensionality reduction (G-611) means reducing the number of features in the data while keeping as much of the useful information as possible. Its two kinds are the last two parts of feature engineering, taught in **feature selection** (G-768) and **feature extraction** (G-762):

- **Feature selection** keeps a subset of the existing features unchanged, for example with forward selection (add the most useful feature, one at a time) or backward elimination (drop the least useful feature, one at a time).
- **Feature extraction** builds new features, each a mix of all the old ones, so that a few new features hold most of the information.

Figure 6 shows both on five features, F1 to F5: selection keeps F1 and F3, while extraction builds two new features, PC1 and PC2 (the new, mixed features; PC stands for principal component), neither equal to any original feature. The main extraction techniques:

- **PCA** (principal component analysis, G-1469), covered next;
- **LDA** (linear discriminant analysis, G-1059);
- **t-SNE** (t-distributed stochastic neighbour embedding).

PCA, LDA and t-SNE are taught in later Notes.

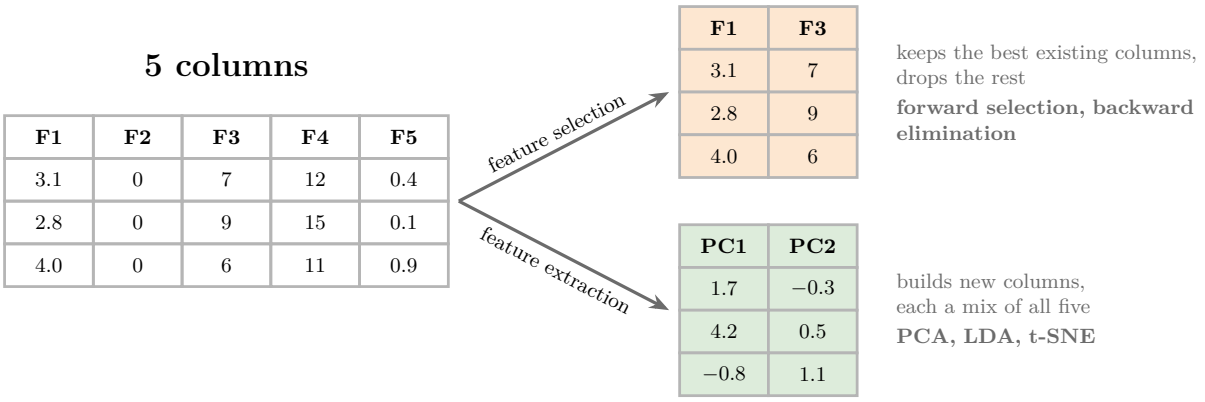


Figure 6: Feature selection vs feature extraction

> **Extra:** The F2 feature in Figure 6 is 0 in every **observation** (G-1374), like the edge pixels in Figure 1. Removing such a constant feature is the simplest feature selection there is. In scikit-learn it is done by `VarianceThreshold`.

7. Summary

	Feature selection	Feature extraction
What it does	keeps the best existing features	builds new features from all the old ones
Output features	a subset of the originals	new, mixed features
Techniques	forward selection, backward elimination	PCA, LDA, t-SNE
When	usually after a first model is built	next Notes (PCA)

- Selection keeps the original **features** (G-772), so the model stays easy to read; extraction mixes all the old features, so a few new ones can hold most of the information.
- Each feature of the data is a dimension, so a table with 1,000 columns is 1,000-dimensional data.
- Every dataset has an **optimal number of features** (G-1397); past it, extra features hurt, so adding columns is not free: on the digits, 400 random features cut accuracy from 96% to 80%.
- More dimensions spread the same data thinner: cells grow as 5^d, points stay the same, so most cells end up empty (86% with 3 features).
- Sparse data makes distances less useful, which hurts distance-based algorithms like KNN, because the nearest neighbour is no longer near and more often shows a different digit.
- Two problems: lower performance and more computation, because every extra feature pushes similar points apart and is more numbers to process.
- The fix is **dimensionality reduction** (G-611): feature selection or feature extraction, because bringing the features down to the optimal number fixes both problems at once.

8. Sources

Built from

- CampusX, "Curse of Dimensionality", YouTube, https://www.youtube.com/watch?v=ToGuhynu-No

Other references

- ESL: Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. §2.5, Local Methods in High Dimensions.
- Beyer, K., Goldstein, J., Ramakrishnan, R. and Shaft, U. (1999). When is "nearest neighbor" meaningful? *Proceedings of the International Conference on Database Theory (ICDT)*, 217–235.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Curse of dimensionality (G-520)	The problems that appear when data has too many dimensions: lower performance and more computation.
High-dimensional data (G-897)	Data with a very large number of columns.
Optimal number of features (G-1397)	The number of columns at which a model performs best.
Dimensionality reduction (G-611)	Reducing the number of input columns while keeping the information.
Feature selection (G-768)	Keeping only the useful input columns and dropping the rest.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Feature extraction (G-762)	Letting an algorithm such as PCA produce new columns from the existing ones (compare feature construction, where we make them by hand).
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Data leakage (G-535)	Information from the test rows reaching the model during training, for example when a scaling or feature-selection step is fitted on all the data before the split; the test scores then come out higher than the model will really do on new data.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Normalization (feature scaling) (G-1349)	The type of feature scaling that squeezes values into a fixed range, such as 0 to 1.
PCA (principal component analysis) (G-1469)	A way to replace many columns with a few new ones that keep most of the spread in the data, using no labels (unsupervised feature extraction for dimensionality reduction). Each new column, a principal component, follows one direction of greatest variance. Used to cut the number of features and to plot data with many columns.

Term	Meaning
LDA (G-1059)	Linear discriminant analysis: a way to build new columns that uses the class labels (a supervised method), along the directions that best separate the classes.
Sparse data (G-1840)	Two senses: a table that is mostly zeros (as after one-hot encoding); or, in many dimensions, a space where most regions hold no points.